23. Flash Attention

25 March 2026 17:49

Transformers → slow & memory hungry on long sequences

Complexity $O(n^2)$

HBM – high Bandwidth Memory → large, main memory on GPU where your model weights and data are stored

SRAM – Static RAM → Much smaller, but significantly faster "on-chip" memory where actual calculations happen

Approximate attention methods → trade of model quality to reduce compute complexity but often do not achieve wall clock speedup

"A missing principle is making attention algorithms IO-aware – accounting for reads and writes between levels of GPU memory"
↓ input/output

Standard attention is memory bound, the processor spends more time waiting for data to arrive from HBM than actually performing calculations

↳ focus on reducing the no of math operations (FLOPS) but real bottleneck isn't the math – its the speed of memory

FLASH ATTENTION → an IO aware exact algorithm that uses tiling to reduce the no of memory reads/writes bwn HBM & GPU

↳ requires fewer HBM accesses than standard attention

↳ optimal for a range of SRAM sizes

They also extent Flashattention to block-sparse attention, yielding an approximate attention algorithm → faster than any existing approximate attention method
↓
Trains transformers faster than existing baselines

Flash Attention & its block-sparse extension enable longer context in transformers, yielding higher quality models and new capabilities

equipping longer context with transformers remain difficult

Make Attention fast & memory efficient → will it address models runtime & memory challenges for long sequences

↳ Approximate attention methods tried it
(sparse-approximation, low rank approximation and their combinations)

↓ These reduce compute requirements to linear or near linear in seq length → but do not display wall clock speed up

↓ the focus on FLOPs reduction (which may not correlate with wall clock speed) & tend to ignore over heads from memory access

Compute bound
→ time accessing HBM is less dominant
→ compute dominates total time

→ They avoid storing full attention matrix

Instead store suff-cient statistics
(like normalization terms)
recompute when needed

* Compute speed has out paced memory speeds and most operations in Transformers are bottlenecked by memory accesses

Flash attention → computes exact attention with far fewer memory accesses

Goal to avoid reading & writing the attention matrix to and from HBM

requires (i) computing the softmax reduction without access to the whole input

(ii) Not storing the large intermediate attention matrix for backward pass

large intermediate matrices are involved in Attention

They do not fit in SRAM

So they must be written to HBM anyway

Soft max couples?
(each output i depends on all elements)

Techniques (i) Restructure attention computation to split the input into blocks & make several passes over input blocks thus incrementally performing the softmax reduction (also known as tiling)

(ii) We store the softmax normalization factor from the forward pass to quickly recompute attention on chip in the backward pass, which is faster than the standard approach of reading the intermediate attention matrix from HBM

If you split into blocks
you cannot compute softmax independently per block

Because denominator = Global sum over all blocks

flash attention implementation on CUDA for fine grained control over memory access, fusing all attn operations into one CUDA kernel

↳ even with increased FLOPs due to recomputation, Algo runs faster & uses less memory – linear in sequence length

↳ $O(N^2 d^2 M^{-1})$ HBM accesses $d \rightarrow$ head dimension
$M \rightarrow$ size of SRAM

Compared to $\Omega(Nd + N^2)$ of standard attention

They store
$m(x)$ = max value for numerical stability
$\ell(x)$ = sum of exponentials (normalization)

These allow computing softmax incrementally across blocks

They are suff-cient statistics for softmax

They provide a lower bound, showing that no exact attention algo can asymptotically improve on the HBM accesses over all SRAM sizes

block-sparse Flashattention, a sparse attention algorithm that is 2-4x faster than flashattention

Correct order
Compute block

Don't store S, P
↓

improve on the HBM accesses over all SRAM sizes

block-sparse Flash attention, a sparse attention algorithm that is 2-4x faster than flash attention
Scaling upto 64K sequence length

block sparse variant has better IO complexity by a factor proportional to sparsity ratio

Flash Attention → Faster Model Training
Higher Quality Models
Benchmarking Attention

GPUs → HBM → SRAM
slower / larger smaller / faster by an order of magnitude

GPUs have a massive no of threads to execute an operation (called a kernel)
Each kernel loads an inputs from HBM to registers and SRAM computes and writes outputs to HB

Performance characteristics Depending on the balance of computation and memory accesses

Operations → memory bound
→ compute bound

Measured by arithmetic intensity
(no of arithmetic operations per byte of memory access)

① compute bound The time taken by the operation is determined by how many arithmetic operations there are.
while time accessing HBM is much smaller eg matrix mult, convolution with large no of channels

② Memory bound time taken determined by no of memory accesses, while time spent in computation is much smaller
eg element wise (activation, drop out)
reduction (sum, softmax, batchnorm, layernorm)

Kernel fusion → Most common approach to accelerate memory bound operations
if there are multiple operations applied to same input
the input can be loaded once from HBM, instead of multiple times for each operation

Standard attention $Q, K, V \in R^{N \times d}$ N-seq length

$S = QK^T \in R^{N \times N}$ $P = \text{softmax}(S) \in R^{N \times N}$

$O = PV \in R^{N \times d}$

Standard attention materialize S, P to HBM makes $O(N^2)$ memory

often $N >> d$

As some or most of operations are memory bound, the large no of memory accesses translate to slow wall clock time (other operations like masking worsens)

Main idea
↳ split inputs Q, K, V into blocks
↳ load them from slow HBM to fast SRAM
↳ then compute the attention output w.r.t those blocks
↳ Scale the output of each block by the right normalization factor before adding them up, we get correct result at the end

Tiling We compute attention by blocks
softmax couples columns of K, so we decompose large softmax with scaling

for numerical stability
softmax of vector $x \in R^B$ is computed as

$m(x) = \max_i x_i$

Correct order
Compute block
Update stats (m, l)
Combine on the fly

Don't store S, P
↓
recompute using stats

Algo ①
→ two nested loops

Outer loop → over K, V blocks
Inner loop → Over Q blocks

They never materialize the full N×N attention matrix

Algo –
Require Q K V $R^{N \times d}$ in HBM
1 Load Q, K by blocks from HBM
compute $S = QK^T$, write to HB
2 Read S from HBM compute $P = \text{softmax}(S)$
write P to HBM
3 Load P & V by blocks from HBM compute
$O = PV$, write O to HBM
4 return

Problem e^{x_i} can overflow if x_i is large

fix shift inputs $e^{x_i} \to e^{x_i - m(x)}$

Same result, numerically stable

$m(x)$ is used for numerical stability, not modeling

$m(x)$, $l(x)$ can be updated incrementally across blocks

Why K, V outer loop?

We want to load K, V block once into SRAM
Re use it as much as possible
Avoid reloading it from HBM

Forward pass = Streaming computation
States are updated online

for numerical stability

Softmax of vector $x \in \mathbb{R}^B$ is computed as

$$m(x) = \max_i x_i$$

$$f(x) = [e^{x_1 - m(x)} \quad e^{x_B - m(x)}]$$

$$l(x) = \sum_i f(x)_i$$

$$\text{Softmax} = \frac{f(x)}{l(x)}$$

for vectors $x^{(1)} x^{(2)} \in \mathbb{R}^B$

We can decompose the soft max of the concatenated $x = [x^{(1)} \; x^{(2)}] \in \mathbb{R}^{2B}$ as

$$m(x) = m([x^{(1)} x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)}))$$

$$f(x) = [e^{m(x^{(1)}) - m(x)} f(x^{(1)}), \; e^{m(x^{(2)}) - m(x)} f(x^{(2)})]$$

$$l(x) = l([x^{(1)}, x^{(2)}]) = e^{m(x^{(1)}) - m(x)} l(x^{(1)}) + e^{m(x^{(2)}) - m(x)} l(x^{(2)})$$

$$\text{Softmax} = f(x)/l(x)$$

Therefore if we keep track of some extra statistics $(m(x), l(x))$ we can compute softmax one block at a time

Split inputs Q, K, V into blocks

Compute softmax values along with extra statistics

Combine the results

Recomputation

not store $O(N^2)$ intermediate values for backward pass

Back pass typically requires S, P to compute gradients

By storing O & softmax norm stats (m, l) we can recompute early

{Selective Gradient Checkpointing}

even with more FLOPs because of reduced HBM accesses the algo speeds up

Refer the Paper for Algorithm

Theorem ① Algo returns $O = \text{softmax}(QK^T)$ with $O(N^2 d)$ Flops and requires $O(N)$ additional memory beyond inputs and output

Theorem ② $N \to$ seq length, $d \to$ head dim, $M \to$ SRAM size } with $d \le M \le Nd$

Standard attention requires $\Theta(Nd + N^2)$ HBM accesses flash attention requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses

Proposition ③ There exists no algorithm to compute exact attention with $O(N^2 d^2 M^{-1})$ HBM accesses for all M in the range of $[d, Nd]$

Block-sparse Flash Attention IO complexity smaller than Flash attention by a factor proportional to sparsity

$Q, K, V \in \mathbb{R}^{N \times d}$ Mask matrix $M \in \{0,1\}^{N \times N}$

$S = QK^T$ $P = \text{softmax}(S \odot \tilde{M})$ $O = PV$

$(S \odot \tilde{M})_{kl} = S_{kl}$ if $\tilde{M}_{kl} = 1$

$-\infty$ if $\tilde{M}_{kl} = 0$

We require $\tilde{M}$ to have block form for some block sizes B_r, B_c for all k, l

$\tilde{M}_{kl} = M_{ij}$ with $i = \lceil k / B_r \rceil$, $j = \lceil l / B_c \rceil$ for some $M \in \{0,1\}^{N/B_r \times N/B_c}$

given this we can adapt Algo ① to compute non zero blocks of the attention matrix

Proposition ④ Let N be seq length, $d \to$ head dim } Block sparse Attention requires

Forward pass = Streaming computation

States are updated online

Not a two phase process

Tiling works $\to$ process blocks independently

Softmax $\to$ requires global normalization

When a new block comes

Step 1 update max

$m_{new} = \max(m_{old}, \max(\text{new block}))$

Step 2 rescale old sum

Old l was computed using m_{old}

Now base changed $\to$ must adjust

$l_{old} \to l_{old} \, e^{m_{old} - m_{new}}$

Step 3 Add new contribution

$l_{new} = l_{old}$ (rescaled) $+ \sum e^{x_i - m_{new}}$

Without rescaling

Old and new values are in different exponential scales

Sum becomes incorrect

$m(x)$ tracks the running max

$l(x)$ accumulates the exponentials

$\text{Softmax} = f(x)/l(x)$

for each block

compute local max

update global max

rescale old sum

add new exponentials

Continue

We maintain a running max and a properly scaled running sum of exponentials so that each new block can be incorporated into an exact global softmax

When max increases

old contributions are downscaled

When max stays same

factor = 1 no change

non zero blocks of the attention matrix

Proposition (4) Let N be seq length, d → head dim, M → SRAM size with $d \le M \le Nd$ } Block sparse Attention requires $\Theta(Nd^2 + N^2d^2M^{-1}s)$ HBM accesses where s is fraction of non zero blocks in block sparsity mask

Limitations (i) coupling to CUDA

Write a CUDA Kernel for each new attention implementation

May not be transferable across GPU architectures

(ii) IO aware deep learning

(iii) Multi GPU IO Aware methods

Algorithm - I

$Q, K, V \in \mathbb{R}^{N \times d}$

Output $O \in \mathbb{R}^{N \times d}$

AM $ie = M$

Block sizes $B_c = \frac{M}{4d}$ $B_r = \left(\frac{M}{4d}\right), d$

Step 1 → partitioning

Thy split $Q \to Q_1, Q_2 \quad Q_T$ (row blocks of size B_r)

K, V → column blocks of size B_c

O, l, m → also split accordingly } each block fits in SRAM

Step 2 Initialize

for each row block:

$m_i = -\infty$

$l_i = 0$

$O_i = 0$ } for each query block we build running max, running normalization, running output

Step 3 Outer loop (over K, V blocks)

for each block j

load K_j, V_j → SRAM } this is where KV reuse happens

Step 4 Inner loop (Over Q blocks)

For each Q_i

Load Q_i

Step 5 Compute Scores

$S_{ij} = Q_i K_j^T$ } vectorium defining matrix shape $B_r \times B_c$

Step 6 Row-wise Max

$\tilde{m}_i = \max(S_{ij}, \text{row-wise})$ } meaning Local max for this block

Step 7 Update Global M

$m_i^{new} = \max(m_i, \tilde{m}_i$

Step 8 compute exponentials

$P_{ij} = e^{S_{ij} - m_i^{new}}$

Step 9 Update Normalization

$$l_i^{new} = \underbrace{l_i e^{m_i - m_i^{new}}}_{\text{rescaled old}} + \underbrace{\sum P_{ij}}_{\text{new contribution}}$$

Step 10 Update output

$$O_i^{new} = \frac{O_i l_i e^{m_i - m_i^{new}} + P_{ij} V_j}{l_i^{new}}$$

} Old output → rescaled, new contribution → added, normalize → divide by new l

Step 11 – After finishing inner loop over all Q_i for a fixed K_j, V_j

for each i, we update i l

Why do we multiply $O_i l_i$ before rescaling?

Why divide by $l_{i\,new}$ at the end?

Because we are maintaining

$$O_i = \frac{\sum e^s V}{\sum e^s}$$

But incrementally across blocks

rewrite the eqn

Current stored output =

$O_i = \frac{\text{old numerator}}{l_i}$

Old numerator $= O_i l_i$ } $O_i l_i$ recovers the unnormalized sum

Now the update makes sense

$O_i l_i e^{m_i - m_i^{new}}$ (rescaled to new max)

New contribution $P_{ij} V_j$

Total numerator Old + new

Normalize $\frac{\text{total numerator}}{l_i^{new}}$

* we temporarily move from normalized to unnormalized space, combine and then renormalize

Compressed version

Goal $O = \text{softmax}(QK^T)V$

Break into: Split $Q \to Q_i$

Split $K, V \to K_j, V_j$

Step 11 – After finishing inner loop over all Q_i for a fixed K_j, V_j

for each i we update
$O_i \leftarrow O_i^{new}$
$l_i \leftarrow l_i^{new}$
$m_i \leftarrow m_i^{new}$

for each query block
We have now incorporated this KV block's contribution
and updated → running max
normalization
Output

Step 12 Continue Outer loop

Move to next K_j, V_j block and repeat

Across all j

Each Q_i accumulates contributions from K_1, K_2 K_-

This is equivalent to
attending over the full sequence

Step 13 Final Output

After all KV blocks processed

O_i contains final attention output

Concatenate all O_i → full O

Standard Attention

HBM accesses

Read $Q, K, V \rightarrow O(ND)$

Write $S \rightarrow O(N^2)$

Read S, write $P \rightarrow O(N^2)$

Read P, V, write $O \rightarrow O(ND)$

$O(N^2) \rightarrow$ dominant term

Goal $O = \text{softmax}(QK^T)V$

Break into blocks
Split $Q \rightarrow Q_i$
Split $K, V \rightarrow K_j V_j$

Process One KV block at a time
Stream through Q blocks

What we maintain (per Q_i)

for each query block
$O_i \rightarrow$ running output
$l_i \rightarrow$ running denominator
$m_i \rightarrow$ running max

For each KV block
for each pair (Q_i, K_j)

(a) compute scores $S_{ij} = Q_i K_j^T$

(b) Local max $\tilde{m} = \max(S_{ij})$

(c) Update global max
$m_i^{new} = \max(m_i, \tilde{m}_i)$

(d) Compute exponentials
$P_{ij} = e^{S_{ij} - m_i^{new}}$

(e) Update normalization
$l_i^{new} = l_i e^{m - m_i^{new}} + \sum P_{ij}$

(f) Update output
$$O_i^{new} = \frac{O_i l_i e^{m_i - m_i^{new}} + P_{ij} V_j}{l_i^{new}}$$

Repeat over all KV blocks
Each Q_i accumulates full attention
Concatenate all Q_i

Q blocks → streamed
KV blocks → reused
No S, no P in HBM

HBM accesses $\sim O\left(\frac{N^2 d^2}{M}\right)$

Larger SRAM → fewer HBM accesses
smaller blocks → more reuse → less memory traffic

Theorem 3
↳ No exact attention algorithm can asymptotically use fewer HBM accesses than this (given SRAM size M)

Meaning –
You must process all N^2 interactions
SRAM can only hold M data at once
So data must be moved from HBM → SRAM

Block Sparse Flash Attention

What changes?
Introduce a mask Matrix M
→ same shape as attention matrix
but
Some blocks = zero (ignored)
Some blocks = computed
Instead of all (i,j)
We compute only
$(i,j) \in$ allowed blocks
Attention becomes sparse at block level

Block Structure
Divide attention matrix into
$\frac{N}{B_r} \times \frac{N}{B_c}$
each block either active or skipped

If sparsity is random
needed elements are scattered
You cannot load them as single dense block

↳ many small irregular accesses
poor memory coalescing
low SRAM utilization
} ability to tile efficiently breaks

Block-sparsity works because
computation is still done on dense blocks
each block fits in SRAM
Can be processed with same algorithm

Complexity change
s = fraction of non-zero blocks

Complexity change

s = fraction of non-zero blocks

HBM accesses $\sim s \frac{N^2 d^2}{M}$

Interpretation

smaller $s \rightarrow$ fewer interactions

fewer memory accesses

faster

You beat the lower IO Bound by reducing problem size (effective N^2)